Document Loading and Splitting

Thursday, March 12, 2026 6:04 PM

Recap:

RAG

- Prompt Engg → No need any additional Content
- Fine Tuning → Quality. Low late → Costly
- RAG. → Cost effective → Medium response rem.

what are other options?

Why RAG? ← what problem with LLM → Leads? Hallucination, Reteration

Cutoff data GPT5-nano. 2025-05-07

what are adv of RAG over other options?

- Economical
- No limitation on quantity. data

RAG pipe line ?

↳ Example :

proprietary data → Not publicly Available

1) Indexing → Data → Embedd → vect data

2) Retriving — Remove the very similar Content

3). Generation — Based on user query

User query + Info retrived vector database

↓

LLM

↓

Generate response

↳ Contextually rich

Indexing

↳ Data → Embeddings → Store → vector DB.

1) Data loading

↓

whole data → langchain doc obj

↓

A lot of doc formats. supp by langchain

Txt, pdf, epub, csv, html, wikipedia

2) Data Splitting

↳ Conv lang doc obj

↓ documents

↓ Chunks.

Based on length of Content

(or) Seperator

3) Document Embeddings.

↳ Document (chunk) → Vector embeddings.

4) Storing data

Vector embeddings → Vector Database.

@ Retrieval (0 to 1) → Customer query → vector embedding

V_c

V_1

V_2 0.7

⋮

V_{50} 0.6

⋮

V_{100}

Max Sim b/w V_c & V_i

V_c → V_1, V_2, V_3, ⋮, V_{100}

Vector database: V_1, V_2, ⋮, V_n

query + (V_2, V_{50}) Content LLM

Text, Text

↓

LLM

↓

response

Gpt-5 → text emb3

Gemi - Gemini